

Failure Taxonomy for AI-Assisted Debugging

An evidence-centered classification of execution, diagnostic, repair, verification, context, and governance failures

T. R. Bentley

Failure Analysis Paper | Paper 9

September 2026

Evidence basis. The taxonomy separates empirically observed failure modes from prospective AI-repair failure classes. Its measured layer is grounded in the 24-case frozen execution benchmark from Paper 8: 16 observed failures, 14 with non-empty diagnostic evidence and two native runtime failures with empty evidence.

Companion data: `AI_Debugger_Pro_Paper9_Failure_Taxonomy_Data.csv`

Abstract

AI-assisted debugging systems can fail at several distinct layers: a program may fail to execute, evidence may fail to explain the termination, an AI diagnosis may misidentify the cause, a proposed patch may repair only the visible symptom, verification may accept an inadequate change, project context may be incomplete, or a human operator may authorize a harmful repair. Collapsing these outcomes into a single 'debugging failure' obscures both engineering responsibility and measurable risk.

This paper introduces a layered failure taxonomy for AI Debugger Pro. The taxonomy distinguishes execution failures, evidence failures, diagnosis failures, repair failures, verification failures, context failures, and governance failures. The first two layers are grounded in measured Paper 8 data. Among 16 observed failing executions, all syntax/compile failures and timeouts produced non-empty evidence, while two of six runtime failures - nonzero C and C++ exits with no stderr - produced empty diagnostic text. The remaining taxonomy layers define testable classes for the forthcoming frozen AI-repair benchmark and human-factors study.

1. Why a Layered Taxonomy Is Necessary

A debugger can correctly recognize that something failed while still failing to provide evidence useful enough for diagnosis. Likewise, an AI can correctly diagnose a defect but propose an overbroad patch, or propose a correct patch that a weak verifier rejects. These are not interchangeable errors. A useful taxonomy therefore identifies the stage at which information, reasoning, mutation, verification, or authorization breaks down.

The proposed model is a seven-layer chain: **Execution -> Evidence -> Diagnosis -> Repair -> Verification -> Context -> Governance**. A defect may propagate across layers, but its primary failure class is assigned to the earliest layer whose contract was violated.

2. Empirically Observed Failure Surface

Paper 8's frozen corpus generated 16 observed failures across Python, C, C++, and Java. The matrix below shows how those failures were distributed by operational class and language.

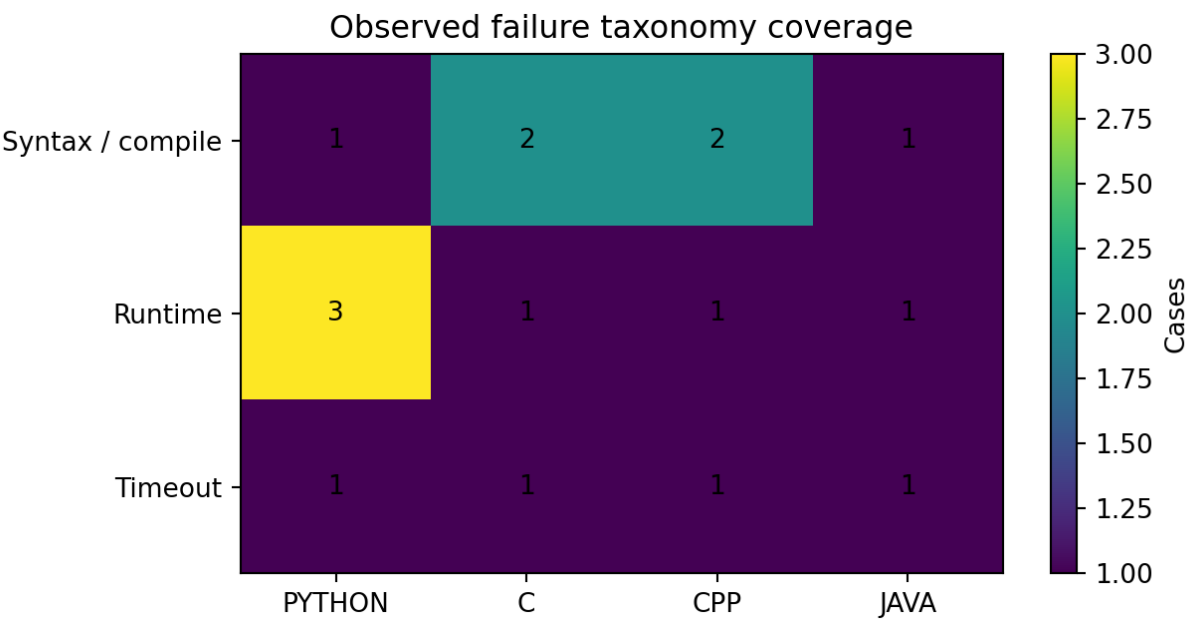


Figure 1. Heatmap of observed failure classes by language. Counts are measured cases, not estimated prevalence in real software.

Failure class	Python	C	C++	Java	Total
Syntax / compile	1	2	2	1	6
Runtime	3	1	1	1	6
Timeout	1	1	1	1	4
Total	5	4	4	3	16

The corpus was designed for coverage rather than prevalence estimation. Therefore, the relative frequency of these categories should not be interpreted as a claim about how often developers encounter them in production.

3. Evidence Failure: The First Measured Taxonomy Defect

Fourteen of the 16 observed failures returned non-empty textual evidence. Two returned an empty string despite a correct failure classification. Both were native runtime cases in which the program deliberately terminated with a nonzero exit status without emitting stderr.

Diagnostic evidence coverage across observed failures (n=16)

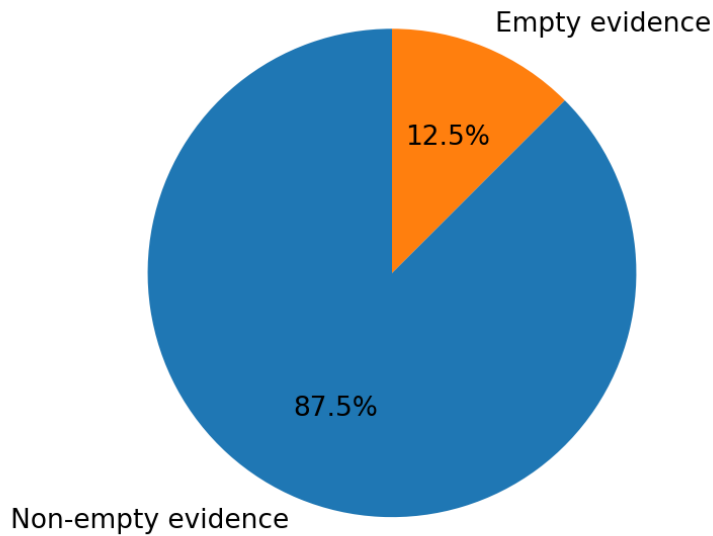


Figure 2. Diagnostic evidence coverage across all measured failures. Non-empty evidence was available for 87.5%; 12.5% exposed an evidence gap.

This motivates a specific taxonomy distinction: **outcome-detection success with evidence-extraction failure**. The executor knew the process failed because the return code was nonzero, but its output contract reduced the failure to stderr text. When stderr was empty, a meaningful machine-level signal was discarded at the human/AI evidence boundary.

A future typed result should preserve exit code, signal where available, timeout status, compilation phase, stdout/stderr presence, backend, language/toolchain, and bounded trace information as independent fields.

4. Seven-Layer Failure Taxonomy

Layer	Failure classes	Primary observable
Execution	Syntax/compile failure; runtime exception; abnormal native exit; timeout; unavailable toolchain; sandbox/backend failure	Program cannot complete as intended.
Evidence	Empty diagnostics; truncated trace; missing exit metadata; misleading stream selection; lost local state	Failure detected but evidence is absent, incomplete, or distorted.
Diagnosis	Wrong root cause; wrong localization; unsupported causal claim; hallucinated API/dependency; unwarranted confidence	Explanation does not match the defect or evidence.
Repair	Incorrect patch; incomplete patch; overfit patch; suppressive patch; insecure patch; unrelated rewrite; no-patch/abstention	Mutation does not safely restore intended behavior.
Verification	Weak oracle; generated-test self-ratification; regression miss; false success; environment mismatch	Bad repair is accepted or good repair is incorrectly rejected.
Context	Relevant file omitted; irrelevant context overload; stale history; wrong project boundary; secret exposure	Reasoning receives the wrong information set.
Governance	Unsafe approval; beneficial repair rejection; evidence ignored; automation bypasses approval; provenance loss	Human/system control policy produces the wrong authorization outcome.

5. Repair Failure Subclasses

Incorrect repair changes code but fails the hidden behavioral oracle. **Incomplete repair** fixes only part of the defect. **Overfit repair** passes visible examples while failing held-out behavior. **Suppressive repair** hides the symptom - for example by swallowing an exception - without restoring intended semantics. **Insecure repair** restores functionality by weakening a security property. **Unrelated rewrite** introduces unnecessary changes outside the defect scope. **Abstention** is not inherently a failure: it becomes an error only when a sufficiently supported safe repair should have been produced under the study's decision rule.

6. Failure Latency Is Not Failure Quality

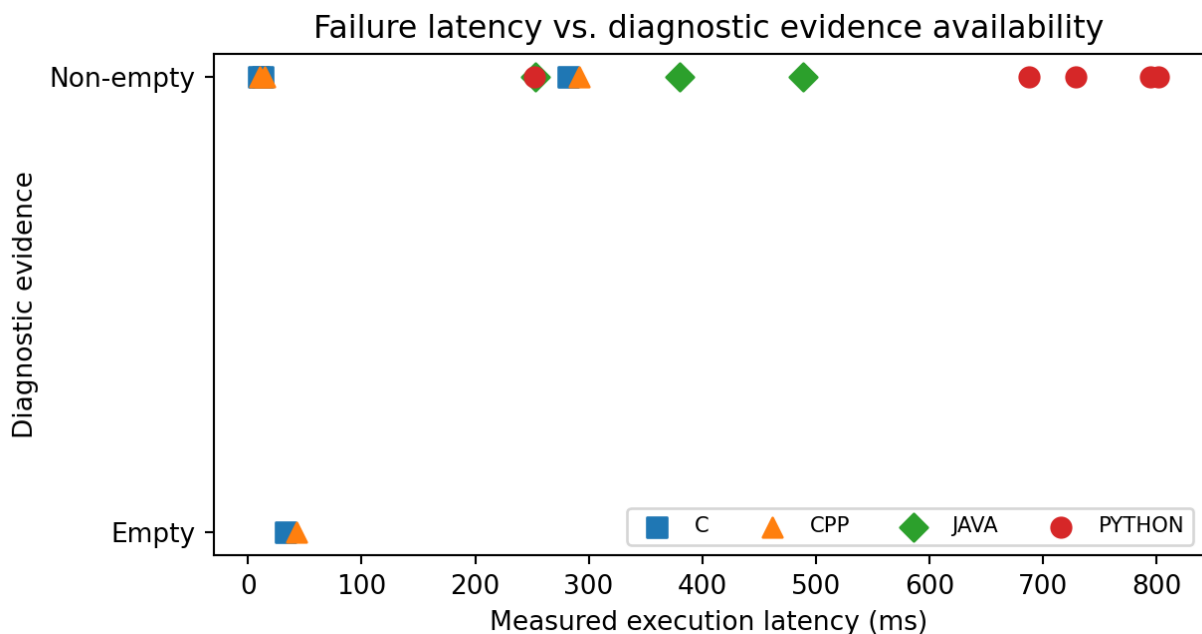


Figure 3. Scatter plot of measured latency against whether diagnostic evidence was non-empty. The two empty-evidence failures were fast native exits, illustrating that fast detection does not imply rich evidence.

The scatter plot prevents an easy but misleading interpretation of performance metrics. A system can return quickly and still return impoverished evidence. Latency, classification correctness, evidence completeness, repair correctness, and verification strength therefore require separate metrics rather than a single aggregate score.

7. Taxonomy-to-Metric Mapping

Failure layer	Recommended measurement
Execution	Classification accuracy; toolchain availability; timeout detection; backend error rate
Evidence	Evidence coverage; typed-field completeness; trace truncation rate; empty-diagnostic rate
Diagnosis	Root-cause accuracy; localization accuracy; unsupported-claim rate; calibration
Repair	Pass@1; hidden-test pass rate; regression-free rate; overfit/suppressive/insecure repair rate
Verification	Verification catch rate; false-verification rate; oracle strength; generated-test agreement with hidden tests
Context	Ablation delta; relevant-file recall; context size; secret leakage rate
Governance	Correct accept/reject rate; unsafe acceptance; beneficial rejection; evidence-request rate; Brier score

8. Classification Rules for Future Benchmarks

Each failed task should receive one primary label and zero or more secondary labels. The primary label identifies the earliest violated contract in the debugging pipeline. For example, a C program returning exit code 3 with empty stderr is primarily an **evidence failure** once the executor has correctly classified the run as unsuccessful. If an AI subsequently invents a root cause because that evidence is missing, **diagnosis failure** becomes a secondary propagated label. This ordering preserves causal structure.

Repairs should be adjudicated against independent hidden tests rather than tests authored by the same model that generated the patch. Security-sensitive tasks additionally require explicit security invariants. Ambiguous cases should permit an 'insufficient evidence' disposition rather than forcing a false binary judgment.

9. Threats to Validity

The measured portion of this taxonomy is based on a small synthetic corpus and should not be treated as a prevalence study. Only execution and evidence layers currently have task-level empirical observations. Diagnosis, repair, verification, context, and governance classes are prospective categories derived from the system architecture and evaluation protocol; their frequencies remain unknown until the corresponding experiments are run. Some defects can legitimately receive multiple labels, so the earliest-contract rule is a methodological convention rather than a claim that software failures always have a single cause.

10. Research Implications

The taxonomy turns future negative results into useful engineering evidence. A lower repair score, for example, becomes substantially more informative when failures can be separated into wrong diagnosis, overfit patch, suppressive patch, insecure patch, or verification miss. This also allows improvements to be evaluated locally: richer executor metadata should reduce evidence failures; better context selection should reduce context-induced diagnosis failures; stronger hidden oracles should reduce false verification; and explicit approval interfaces should be evaluated against governance errors rather than raw model accuracy.

11. Conclusion

AI-assisted debugging reliability is not one number. It is a sequence of contracts connecting execution, evidence, diagnosis, repair, verification, context, and human authorization. Paper 8 supplied the first measured example of why this separation matters: all 16 failing executions were detected, but two native runtime failures lost explanatory evidence at the output boundary. Paper 9 formalizes that distinction and extends it into a taxonomy suitable for the next repair, context-ablation, reproducibility, and human-governance experiments.

Data Availability

The companion CSV aggregates the measured Paper 8 failure observations by language and failure class and records non-empty versus empty diagnostic evidence. The underlying task-level benchmark remains available with Paper 8.